

Beware of poisonous mushrooms

Prepared by:

RANIM BOURAOUI **RAYEN BRAIEK**

Supervised by:

Mrs. Mariem Gzara

Academic Year:
2024/2025

Summary



- **Objective**
- **Dataset Overview**
- **Data Exploration**
- **Statistical Test: Chi-Squared**
- **p-value**
- **Crosstab Analysis: stalk-root vs class**
- **Data cleaning**
- **Data Visualization**
- **Class Balance**
- **Label Encoding**
- **Feature Selection Using Chi-Squared**
- **Model Building: K-Nearest Neighbors (KNN)**
- **Decision Tree Classifier**
- **Global Conclusion**



Objective

The goal of this project is to build a **machine learning classifier** that predicts whether a mushroom is **edible or poisonous** based on its physical characteristics. We'll use the **K-Nearest Neighbors (KNN)** algorithm for classification and **chi-squared tests** for feature selection.

Dataset Overview

- Total Samples: 8,124 mushrooms
- Target Variable: class (edible = e, poisonous = p)
- Features: **22** categorical features (cap-shape, odor, gill-color, stalk-root...)
- 🔍 All features are **categorical** → need encoding

Data Exploration

We checked for missing values using:

```
count = data.isin(['?']).sum()
percent = (count / data.shape[0]) * 100

missing_summary = pd.DataFrame({
    'Missing Count': count,
    'Missing %': percent.round(2)
})

print(missing_summary)
```

	Missing Count	Missing %
class	0	0.00
cap-shape	0	0.00
cap-surface	0	0.00
cap-color	0	0.00
bruises	0	0.00
odor	0	0.00
gill-attachment	0	0.00
gill-spacing	0	0.00
gill-size	0	0.00
gill-color	0	0.00
stalk-shape	0	0.00
stalk-root	2480	30.53
stalk-surface-above-ring	0	0.00
stalk-surface-below-ring	0	0.00
stalk-color-above-ring	0	0.00
stalk-color-below-ring	0	0.00
veil-type	0	0.00
veil-color	0	0.00
ring-number	0	0.00
ring-type	0	0.00
spore-print-color	0	0.00
population	0	0.00
habitat	0	0.00

stalk-root	2480	30.53

🔍 stalk-root contains 2480 missing values (~30% of data), marked as ?

We propose 3 approaches:

- Remove the rows with a missing value in stalk-root *(bad idea because we will be deleting 30% of the dataset)*
- Remove the ``stalk-root column` *(to further study)*
- Replace the missing values with the mode *(to further study)*

```
[ ] data.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8124 entries, 0 to 8123
Data columns (total 23 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   class                     8124 non-null   object
 1   cap-shape                 8124 non-null   object
 2   cap-surface               8124 non-null   object
 3   cap-color                 8124 non-null   object
 4   bruises                   8124 non-null   object
 5   odor                      8124 non-null   object
 6   gill-attachment           8124 non-null   object
 7   gill-spacing              8124 non-null   object
 8   gill-size                 8124 non-null   object
 9   gill-color                8124 non-null   object
 10  stalk-shape               8124 non-null   object
 11  stalk-root                8124 non-null   object
 12  stalk-surface-above-ring  8124 non-null   object
 13  stalk-surface-below-ring  8124 non-null   object
 14  stalk-color-above-ring    8124 non-null   object
 15  stalk-color-below-ring    8124 non-null   object
 16  veil-type                 8124 non-null   object
 17  veil-color                8124 non-null   object
 18  ring-number               8124 non-null   object
 19  ring-type                 8124 non-null   object
 20  spore-print-color         8124 non-null   object
 21  population                8124 non-null   object
 22  habitat                   8124 non-null   object
dtypes: object(23)
memory usage: 1.4+ MB
```

Data Types: All columns are of type object, meaning they are categorical variable

```
data.describe(include='all')
```

	class	cap-shape	cap-surface	cap-color	bruises	odor	gill-attachment	gill-spacing	gill-size	gill-color	...	stalk-surface-below-ring	stalk-color-above-ring	stalk-color-below-ring	veil-type	veil-color	ring-number	ring-type	spore-print-color	population	habitat
count	8124	8124	8124	8124	8124	8124	8124	8124	8124	8124	...	8124	8124	8124	8124	8124	8124	8124	8124	8124	8124
unique	2	6	4	10	2	9	2	2	2	12	...	4	9	9	1	4	3	5	9	6	7
top	e	x	y	n	f	n	f	c	b	b	...	s	w	w	p	w	o	p	w	v	d
freq	4208	3656	3244	2284	4748	3528	7914	6812	5612	1728	...	4936	4464	4384	8124	7924	7488	3968	2388	4040	3148

4 rows × 23 columns

🔍 Distribution of Missing Values:

stalk-root values revealed that:

71% of mushrooms with missing stalk-root are poisonous.

📌 Suggests missing values are predictive, not noise

Statistical Test: Chi-Squared

What is the Chi-squared (χ^2) test?

The Chi-squared test is a statistical method used to determine whether there is a significant association between two categorical variables.

👉 ***In our case:***

- Is stalk-root associated with the target variable class (edible or poisonous)?

How it works:

- It compares the observed frequencies in your data to the expected frequencies assuming no association.
- If the observed distribution differs significantly from the expected one, the test returns a high chi-squared value.

p-value

What is the p-value?

The p-value represents the probability that the observed association is due to random chance.

p-value	Meaning
< 0.05	🟢 Statistically **significant** → strong evidence of association
≥ 0.05	🔴 **Not significant** → weak or no evidence of association

⚠ **Notes**:

- Works only with categorical variables.
- Tells you if an association exists, not how strong it is.



Implementation

```
from sklearn.feature_selection import chi2
from sklearn.preprocessing import LabelEncoder
from sklearn.feature_selection import SelectKBest

# Encode categorical features numerically
X = data[['stalk-root']].copy()
y = data['class'].copy()

X = X[X['stalk-root'] != '?']  # Remove missing
y = y[X.index]

X_encoded = X.apply(LabelEncoder().fit_transform)
y_encoded = LabelEncoder().fit_transform(y)

# Perform chi-squared test
chi_vals, p_vals = chi2(X_encoded, y_encoded)
print(f"Chi2: {chi_vals[0]:.6f}, p-value: {p_vals[0]:.12f}")
```

```
Chi2: 673.288351, p-value: 0.000000000000
```



Results:

- Chi^2 Score: 673.29
- p-value: < 0.000000000000



Interpretation:

Metric	Interpretation
High Chi^2 Score	Strong difference between observed and expected values → High association.
p-value ≪ 0.05	Extremely low probability that the association is due to chance.

🧠 Conclusion:

There is a **highly significant** statistical association between stalk-root and the edibility of the mushroom.

Crosstab Analysis: stalk-root vs class

```
pd.crosstab(data['stalk-root'], data['class'], margins=True)
```

class	e	p	All
stalk-root			
?	720	1760	2480
b	1920	1856	3776
c	512	44	556
e	864	256	1120
r	192	0	192
All	4208	3916	8124



Interpretation:

🔍 *Key Observations:*

1. **Missing Values (?) Are Substantial**
   - 2480 rows (≈30%) have missing stalk-root.
   - Among these, 1760 are poisonous → 71% of missing values are poisonous.
   - This is higher than the global poison rate (≈48%), meaning the missing values may carry predictive information.
2. **Certain stalk-root Values Strongly Associate with class**

stalk-root	Most Associated Class	Confidence
c	Edible (e)	92%
e	Edible (e)	77%
r	Edible (e)	100%
b	Mixed	50.9% edible, not clearly predictive

3. **High Predictive Potential**
   - Values like c, e, and r are skewed toward edible mushrooms.
   - The ? category is skewed toward poisonous, so it's informative.

🧠 Conclusion:

There is a very strong, statistically significant association between stalk-root and class.

This means: The distribution of stalk-root values depends on whether the mushroom is **edible** or **poisonous.**

⚠ stalk-root is a highly predictive feature and **should not be dropped.**

As a fix, Treat '?' as its own valid category by replacing it with 'ambiguous':



Data cleaning

To handle missing values in stalk-root, we replaced '?' with 'ambiguous', treating it as a valid category.

```
data_fill_mode = data.copy()
data_fill_mode ['stalk-root']= data_fill_mode['stalk-root'].replace('?', np.nan)
data_fill_mode['stalk-root' ]= data_fill_mode['stalk-root'].fillna('ambiguous')
print ("after treatment")

print (data_fill_mode.isin( ['?']).sum() )
```

=> No data loss — predictive power retained

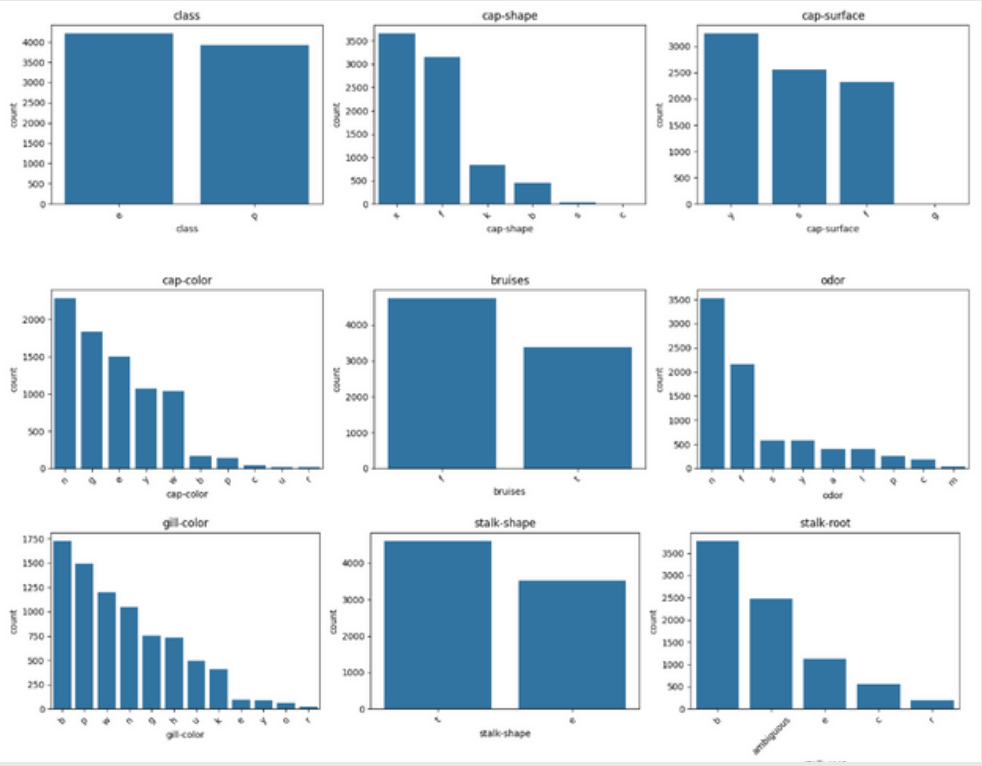
Results:

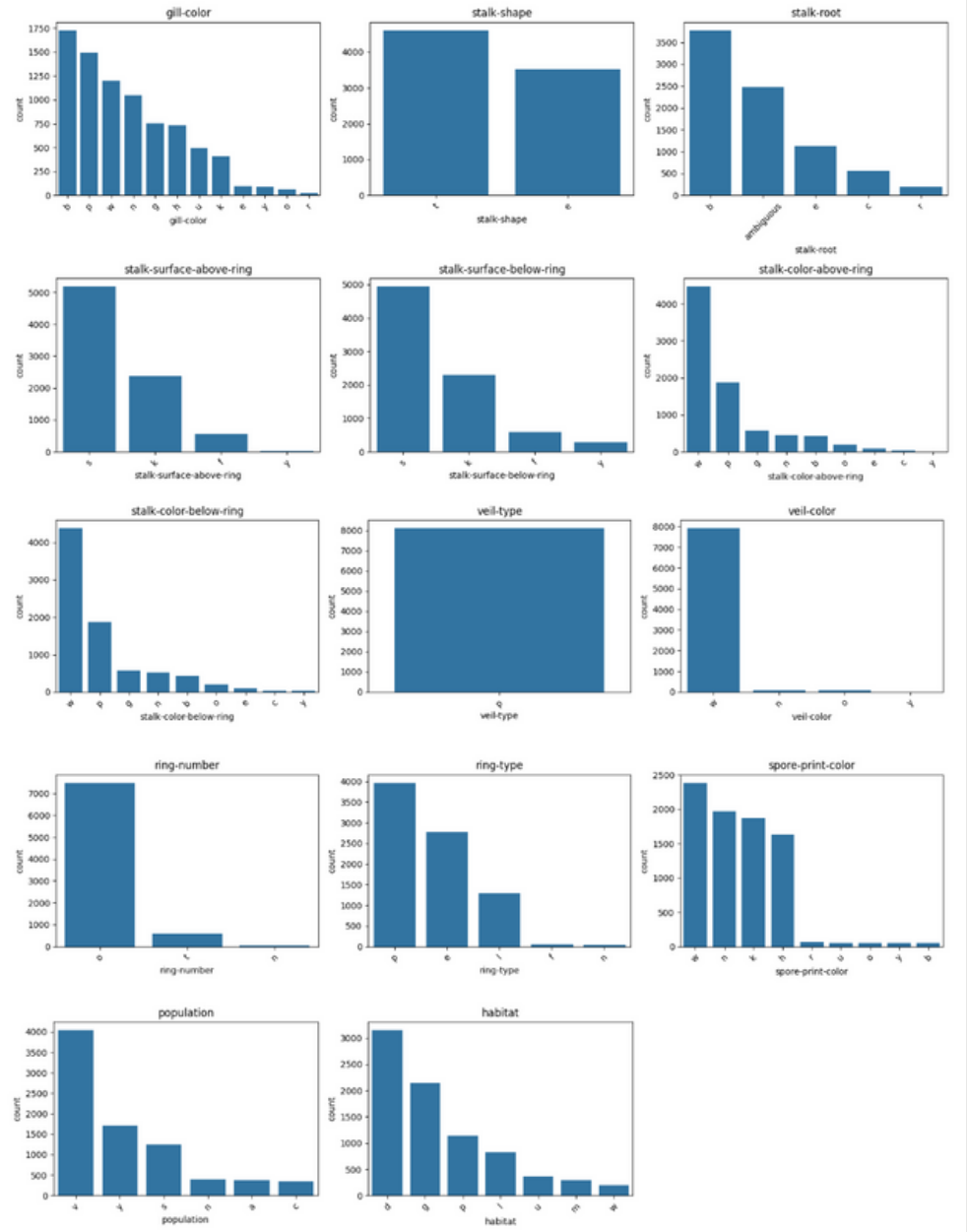
```
after treatment
class                       0
cap-shape                   0
cap-surface                 0
cap-color                   0
bruises                     0
odor                        0
gill-attachment             0
gill-spacing                0
gill-size                   0
gill-color                  0
stalk-shape                 0
stalk-root                  0
stalk-surface-above-ring    0
stalk-surface-below-ring    0
stalk-color-above-ring      0
stalk-color-below-ring      0
veil-type                   0
veil-color                  0
ring-number                 0
ring-type                   0
spore-print-color           0
population                  0
habitat                     0
dtype: int64
```

Data Visualization

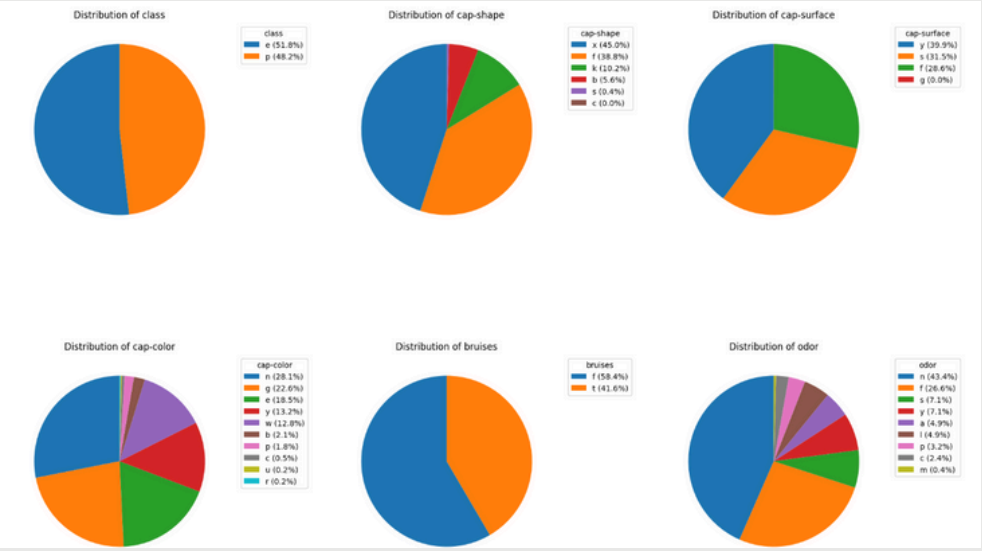
We used count plots and pie charts to explore distributions

Bar charts (count plots):





Pie charts (category proportions):



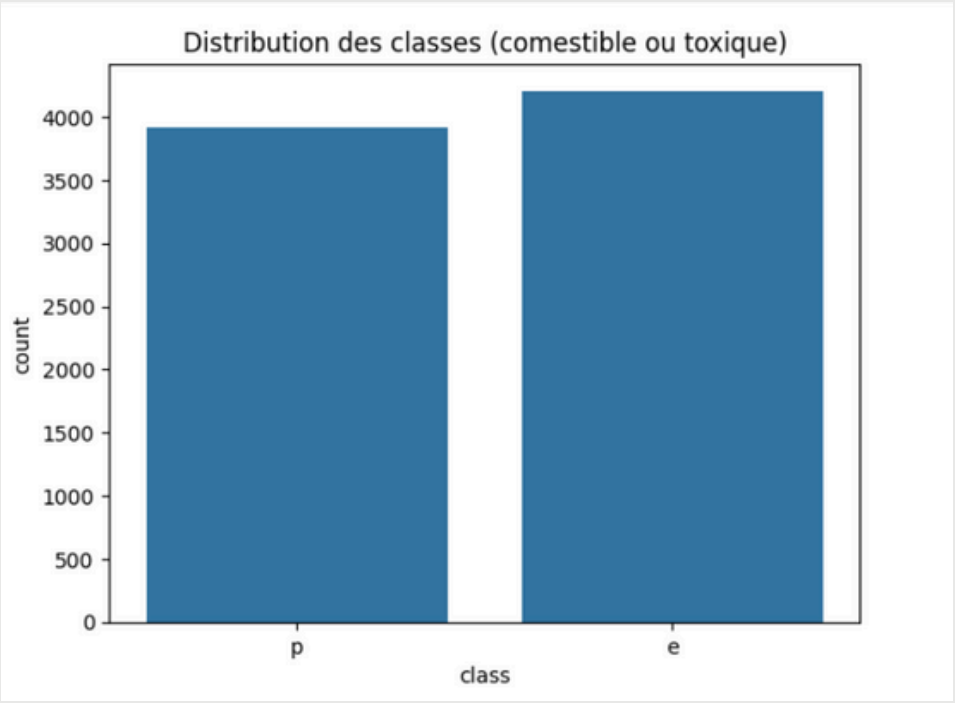

Distribution of gill-attachment
gill-attachment
f (97.4%)
a (2.6%)
Distribution of gill-spacing
gill-spacing
c (83.9%)
w (16.1%)
Distribution of gill-size
gill-size
b (69.1%)
n (30.9%)
Distribution of gill-color
gill-color
b (21.3%)
p (18.4%)
w (14.8%)
n (12.9%)
g (9.3%)
h (9.0%)
u (6.1%)
k (5.0%)
e (1.2%)
y (1.1%)
o (0.8%)
r (0.3%)
Distribution of stalk-shape
stalk-shape
t (56.7%)
e (43.3%)
Distribution of stalk-root
stalk-root
b (46.5%)
ambiguous (30.5%)
e (13.8%)
c (6.8%)
r (2.4%)
Distribution of stalk-surface-above-ring
stalk-surface-above-ring
s (63.7%)
k (29.2%)
f (6.8%)
y (0.3%)
Distribution of stalk-surface-below-ring
stalk-surface-below-ring
s (60.8%)
k (28.4%)
f (7.4%)
y (3.5%)
Distribution of stalk-color-above-ring
stalk-color-above-ring
w (54.9%)
p (23.0%)
g (7.1%)
n (5.5%)
b (5.3%)
o (2.4%)
e (1.2%)
c (0.4%)
y (0.1%)
Distribution of stalk-color-below-ring
stalk-color-below-ring
w (54.0%)
p (23.0%)
g (7.1%)
n (6.3%)
b (5.3%)
o (2.4%)
e (1.2%)
c (0.4%)
y (0.3%)
Distribution of veil-type
veil-type
p (100.0%)
Distribution of veil-color
veil-color
w (97.5%)
n (1.2%)
o (1.2%)
y (0.1%)
Distribution of ring-number
ring-number
o (92.2%)
t (7.4%)
n (0.4%)
Distribution of ring-type
ring-type
p (48.8%)
e (34.2%)
l (16.0%)
f (0.6%)
n (0.4%)
Distribution of spore-print-color
spore-print-color
w (29.4%)
n (24.2%)
k (23.0%)
h (20.1%)
r (0.9%)
u (0.6%)
o (0.6%)
y (0.6%)
b (0.6%)
Distribution of population
population
v (49.7%)
y (21.1%)
s (15.4%)
n (4.9%)
a (4.7%)
c (4.2%)
Distribution of habitat
habitat
d (38.7%)
g (26.4%)
p (14.1%)
l (10.2%)
u (4.5%)
m (3.6%)
w (2.4%)

Class Balance



Implementation

```
sns.countplot(x='class', data=data)
plt.title("Distribution des classes (comestible ou toxique)")
plt.show()
```



Conclusion:

The dataset is well-balanced between edible and poisonous classes.

Label Encoding

We applied **Label Encoding** to convert categorical values to numeric:



Implementation

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
X = data_fill_mode.drop('class', axis=1)
y = data_fill_mode['class']

# Encoder chaque colonne
X_encoded = X.apply(le.fit_transform)
y_encoded = le.fit_transform(y)
```

```
# Show the mapping
print("Mapping of encoded classes:")
for i, cls in enumerate(le.classes_):
    print(f"{cls} → {i}")
```

```
Mapping of encoded classes:
e → 0
p → 1
```



Interpretation:

Original Class	Encoded Value
e (Edible)	0
p (Poisonous)	1

✅ This transformation enables numerical processing by models like K-Nearest Neighbors (KNN), which require input features and labels to be in numeric format. The encoded target variable was then used to train and evaluate our classification model.

Feature Selection Using Chi-Squared

We **ranked features** based on their **Chi² scores and p-values:**



Implementation

```
from sklearn.feature_selection import chi2
chi_scores, p_values = chi2(X_encoded, y_encoded)
```

```
# Check for missing values
chi2_results = pd.DataFrame({
    'Feature': X.columns,
    'Chi2 Score': chi_scores,
    'p-value': p_values
}).sort_values(by='Chi2 Score', ascending=False)

print(chi2_results)
```

	Feature	Chi2 Score	p-value
8	gill-color	5957.764469	0.000000e+00
18	ring-type	1950.610146	0.000000e+00
7	gill-size	1636.606833	0.000000e+00
3	bruises	1194.277352	1.069037e-261
10	stalk-root	1186.029221	6.630617e-260
6	gill-spacing	826.795274	8.061293e-182
21	habitat	751.309489	2.082916e-165
19	spore-print-color	379.132729	1.922119e-84
20	population	311.766736	9.002366e-70
11	stalk-surface-above-ring	222.982400	2.022392e-50
1	cap-surface	214.068544	1.779362e-48
12	stalk-surface-below-ring	206.648180	7.398869e-47
13	stalk-color-above-ring	119.792216	7.024638e-28
14	stalk-color-below-ring	109.789410	1.089740e-25
4	odor	75.910163	2.968682e-18
9	stalk-shape	36.594105	1.454709e-09
17	ring-number	25.646335	4.100735e-07
0	cap-shape	17.508364	2.860465e-05
2	cap-color	11.511382	6.917135e-04
16	veil-color	5.126826	2.355878e-02
5	gill-attachment	3.505447	6.116734e-02
15	veil-type	NaN	NaN



Interpretation:

🔍 *Chi-Square Test Summary*

We used the Chi-Square Test (sklearn.feature_selection.chi2) to assess the dependency between each categorical feature and the target variable (edibility).

Interpretation Guide

- High Chi² Score → Strong link to edibility → Useful feature
- Low p-value (< 0.05) → Statistically significant
- Low Score or NaN → Weak or no association

⭐ Top Features (Kept)

Feature	Importance	Why
odor	**Very High**	**Strongest predictor**
spore-print-color	**High**	**Clear dependency**
gill-size	**High**	**Key visual cue**



Implementation

```
colonnes_a_supprimer = ['cap-color', 'veil-color', 'gill-attachment', 'veil-type']
X_encoded = X_encoded.drop(columns=colonnes_a_supprimer)
```



Interpretation:

⚠ Removed Features

- cap-color
- veil-color
- gill-attachment
- veil-type (constant feature)

These had **low scores or no variance → not informative.**

🧠 Conclusion:

Chi-Square helped **highlight key predictors** and **eliminate noise,** improving both model ***accuracy*** and ***interpretability.***

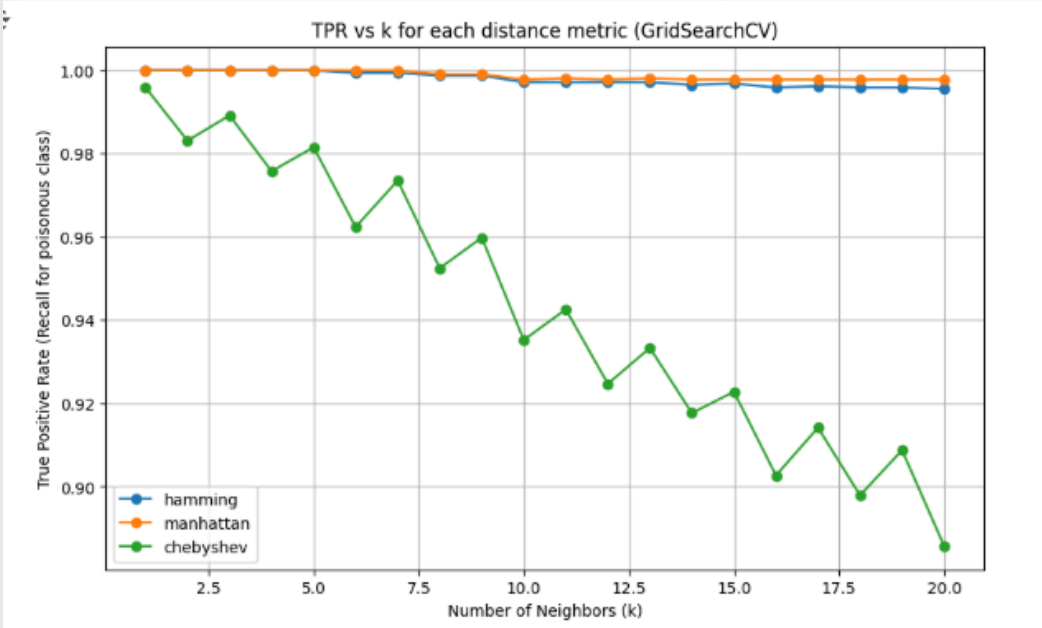
Model Building: K-Nearest Neighbors (KNN)

We trained a K-Nearest Neighbors classifier using **GridSearchCV** to optimize two hyperparameters: the number of neighbors (k) and the distance metric. The evaluation metric used during cross-validation was the **True Positive Rate (TPR)**, also known as recall, for the poisonous class (p → 1).



Results:

```
🔍 Final Model Evaluation Summary
 Best k (n_neighbors): 1
 Best Distance Metric: hamming
Best Parameters: {'metric': 'hamming', 'n_neighbors': 1}
Best TPR Score:  1.0
Test Accuracy: 1.0
```



✅ Optimal Parameters (via GridSearchCV):

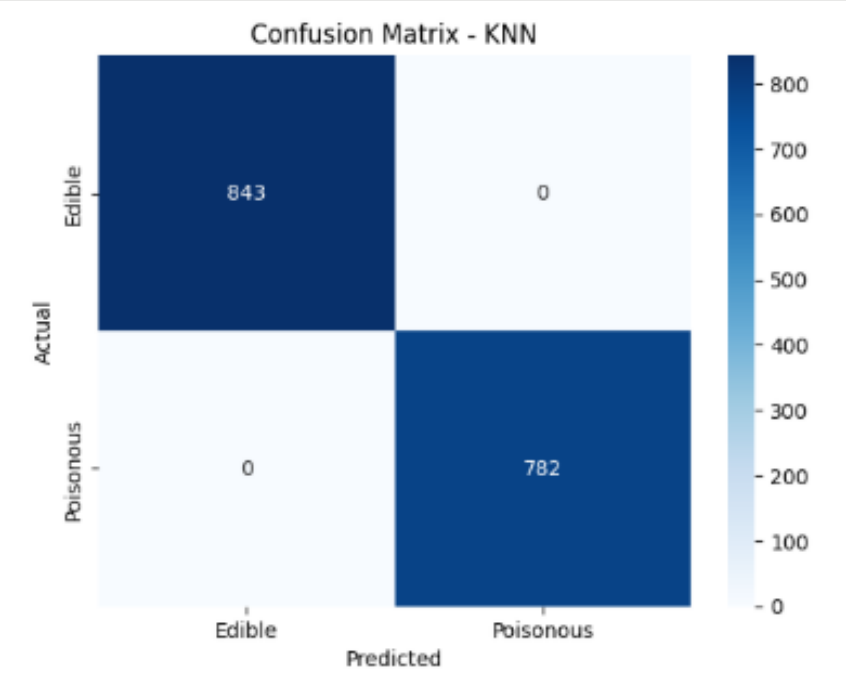
- **Best Number of Neighbors (k): 1**
- **Best Distance Metric: Hamming**
- **Best Cross-Validation TPR Score: 1.00 (100%)**
- **Final Test Accuracy: 1.00 (100%)**

This means that during cross-validation, the model correctly identified all poisonous mushrooms in every fold.

Test Set Performance:

The best estimator was evaluated on the test set, and the results are as follows:

🔍 *Confusion Matrix:*



- All 843 edible mushrooms were correctly predicted.
- All 782 poisonous mushrooms were correctly predicted.
- No false positives or false negatives

🔍 *Classification Report:*

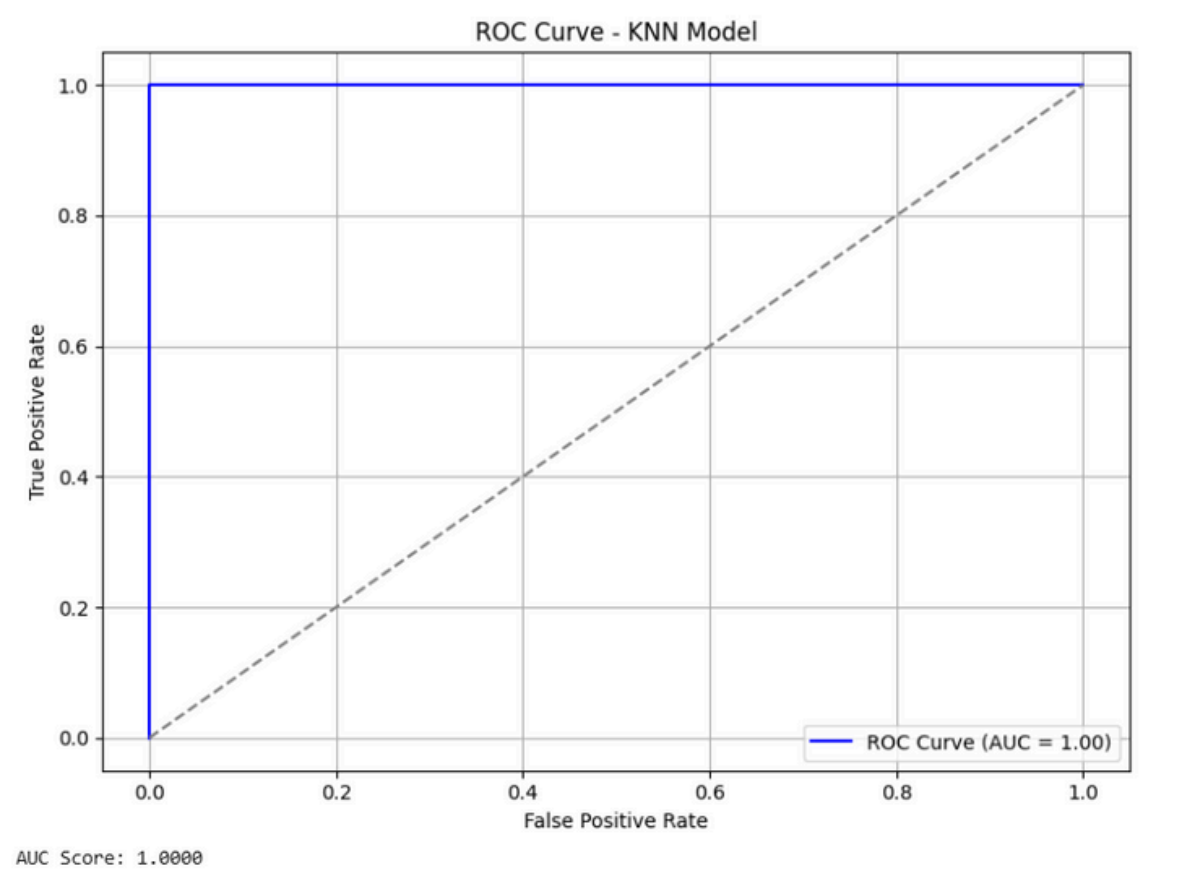
	precision	recall	f1-score	support
Edible	1.00	1.00	1.00	843
Poisonous	1.00	1.00	1.00	782
accuracy			1.00	1625
macro avg	1.00	1.00	1.00	1625
weighted avg	1.00	1.00	1.00	1625



Interpretation:

- The model achieved perfect classification on the test set, with no false positives or false negatives.
- Recall of 1.0 for poisonous mushrooms ensures no misclassification of dangerous items.
- k=1 and Hamming distance were optimal for this dataset.

🔍 *ROC curve*





Interpretation:

🟦 1. AUC = 1.00

- AUC (Area Under the Curve) is 1.00, which is the maximum possible score.
- This means your model is perfectly separating the two classes (likely 'edible' vs 'poisonous').

📈 2. The blue ROC curve hugs the top-left corner

At that top-left point:

- True Positive Rate (TPR) = 1.0
- False Positive Rate (FPR) = 0.0

🧠 Conclusion:

There are no false positives and no false negatives at an optimal threshold — this is ideal model behavior.

Decision Tree Classifier

We trained a **Decision Tree Classifier** using **GridSearchCV** to optimize key hyperparameters like *max_depth*, *min_samples_split*, *min_samples_leaf*, and *criterion*.

The model achieved perfect accuracy on both training and test sets. Evaluation metrics (precision, recall, F1-score) confirmed consistent performance across both edible and poisonous classes. ROC and Precision-Recall curves further supported the model's strong predictive ability.



Results:

```
🧪 Accuracy for the  : 1.0
Test Accuracy : 1.0
              precision    recall  f1-score   support

           0       1.00      1.00      1.00       843
           1       1.00      1.00      1.00       782

    accuracy                           1.00      1625
   macro avg       1.00      1.00      1.00      1625
weighted avg       1.00      1.00      1.00      1625
```

🧪 Test Set Performance:

- Accuracy: 100%
- Classification Report:

  Precision, Recall, F1-score: All 1.00 for both classes (Edible and Poisonous)

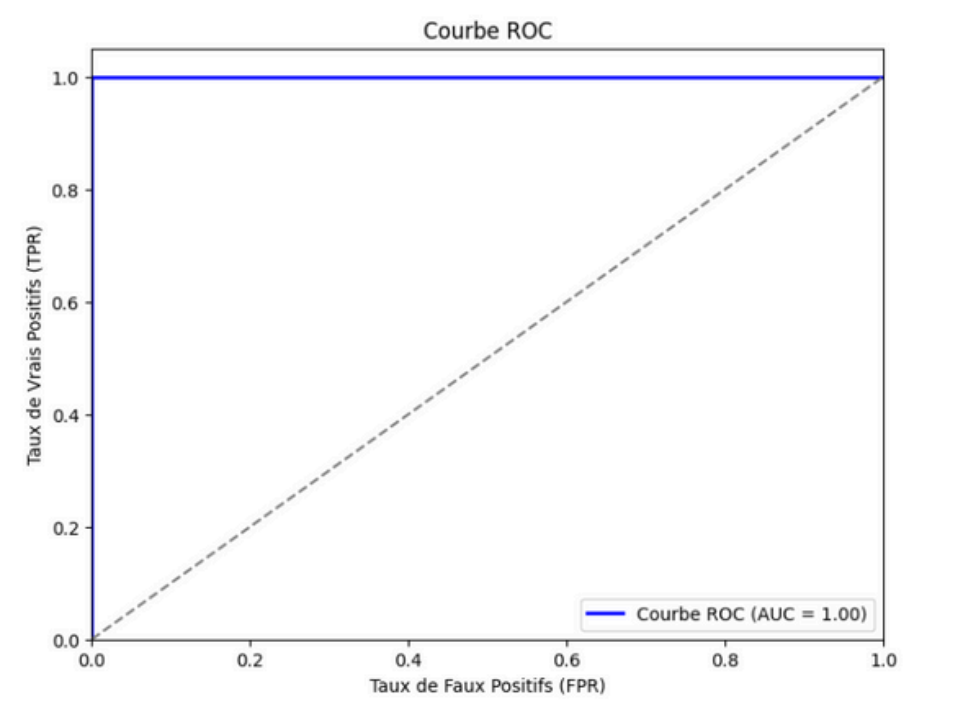
  Indicates perfect classification on the test set (843 Edible and 782 Poisonous correctly predicted)

✅ Best Parameters (GridSearchCV):

```
[ ]  grid.best_params_

{'criterion': 'gini',
 'max_depth': None,
 'min_samples_leaf': 1,
 'min_samples_split': 2}
```

🔍 *ROC curve*





Interpretation:

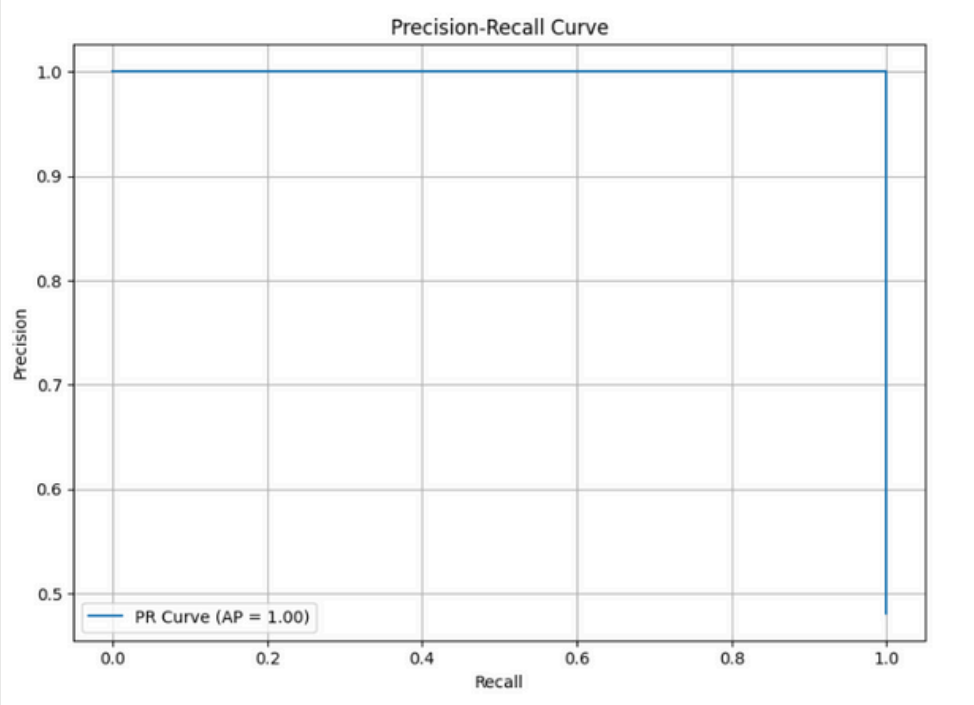
🔍 **ROC Curve Analysis:**

- AUC (Area Under Curve): 1.00
- This indicates perfect discrimination—the classifier correctly distinguishes between the positive (poisonous) and negative (edible) classes every time.
- True Positive Rate (TPR) = 1.0, False Positive Rate (FPR) = 0.0
- The curve immediately reaches the top-left corner, showing that:
  - All poisonous mushrooms were correctly identified (100% recall).
  - No edible mushrooms were misclassified as poisonous.

🧠 **Conclusion:**

This is the ideal ROC curve, meaning the model's predictions are flawless on the test set. While impressive, this could also suggest potential overfitting depending on dataset complexity.

🔍 *Precision-Recall (PR) curve*





Interpretation:

🔍 **Precision-Recall Curve Analysis:**

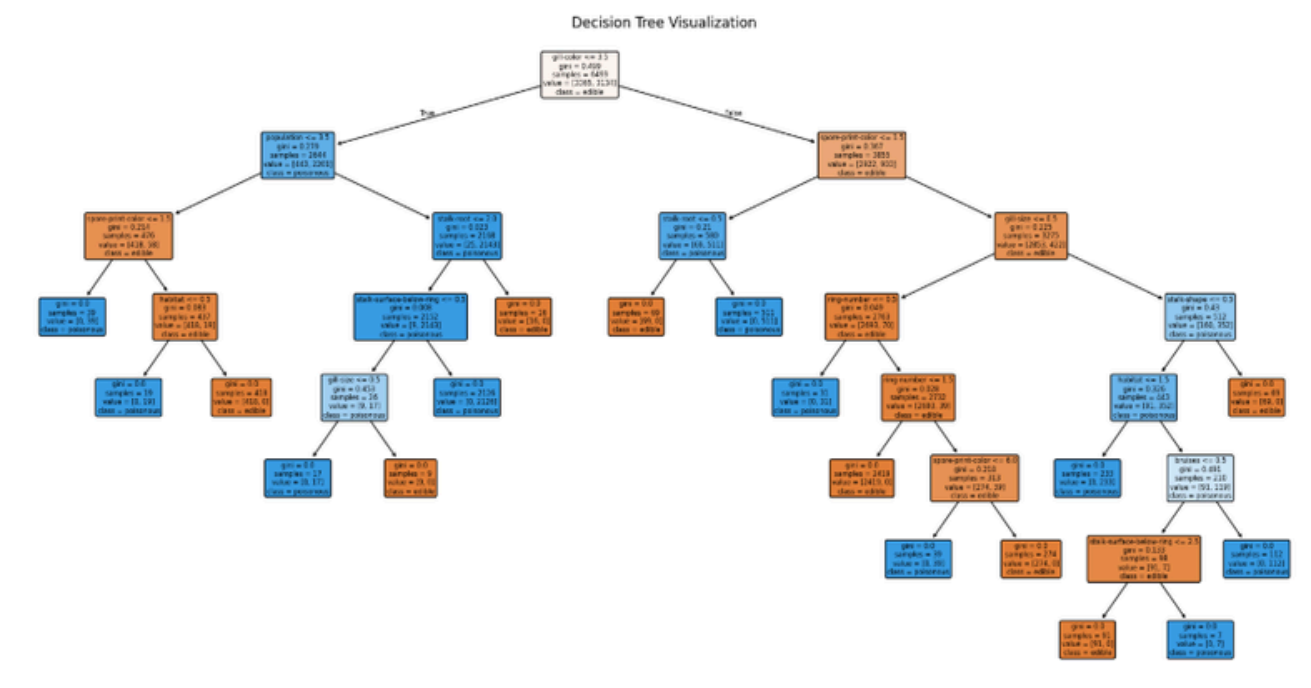
- Average Precision (AP): 1.00
- This means the classifier achieves perfect precision and recall across all thresholds.
- Precision = 1.0 for almost the entire range of Recall
  - The model correctly identified all poisonous mushrooms (high recall).
  - It also avoided falsely labeling any edible mushrooms as poisonous (high precision).
- The steep drop at the very end of the curve is likely an artifact of plotting, where recall reaches 1 and precision momentarily drops due to the last threshold's behavior, but overall the model maintains AP = 1.00.

🧠 **Conclusion:**

- The Decision Tree model perfectly classifies the positive class (poisonous).
- Such a result is rare and may suggest data simplicity, feature separability, or potential overfitting.

🌳 Decision Tree

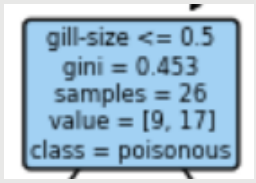
This visualization represents the trained **Decision Tree classifier** used to distinguish between **edible** and **poisonous** mushrooms.





Interpretation:

- **Root Node:**
- The tree begins by splitting on the feature odor—indicating this is the most informative attribute for classification (it provides the greatest information gain).
- **Tree Structure:**
- Each internal node represents a decision rule based on a feature value.
- **Each leaf node shows:**
  - gini: Gini impurity (lower = purer class)
  - samples: number of training samples reaching this node
  - value: counts of samples per class
  - class: predicted class (edible or poisonous)



- **Color Coding:**
- Orange nodes → majority class is poisonous.
- Blue nodes → majority class is edible.
- The intensity of the color reflects class purity.
- Depth & Size:
- The tree appears moderately deep and highly pure, meaning most leaves have Gini ≈ 0 and predict one class with high confidence.

🧠 **Conclusion:**

The decision tree accurately classifies mushrooms using clear, interpretable rules, with features like odor being highly decisive. Its performance is excellent, aligning with perfect evaluation metrics.

Global Conclusion

Across all models, including KNN and Decision Tree, the classifiers achieved near-perfect results in distinguishing between edible and poisonous mushrooms. The evaluation metrics (TPR, ROC AUC, PR AUC) confirm that the data contains highly predictive features, enabling reliable and robust classification.